

Exercise Sheet 8

Adversarial Machine Learning

Adversarial Attacks

Exercise 8.1: What Are Adversarial Examples?

optional

Explain what an adversarial example is. How do they differ from Out-of-Distribution examples?

An **adversarial example** is an input to a machine learning model that has been intentionally and subtly modified to cause the model to make a mistake or misclassification. These modifications (perturbations) are often completely imperceptible to human eyes but are highly disruptive to the model's decision-making process.

Difference from Out-of-Distribution (OOD) Examples

- **Adversarial Examples:** These originate from the *same underlying distribution* as the training data but are deliberately optimized (often using gradient information) to cross a nearby decision boundary. They represent worst-case, targeted manipulations.
- **Out-of-Distribution (OOD) Examples:** These are clean, unmanipulated inputs that come from a *completely different distribution* than what the model was trained on. For instance, showing an image of an airplane to a model trained exclusively to distinguish dogs from cats is an OOD example. They represent a shift in the environment rather than a malicious manipulation.

Exercise 8.2: Attack Formulation

A basic gradient-based attack updates the input as:

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \alpha \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{y}, f(\mathbf{x}_i)) \quad (1)$$

1. What does each term represent?

1. Representation of Terms

- $\mathbf{x}_i$: The input vector (e.g., an image) at the current iteration i .
- $\mathbf{x}_{i+1}$: The updated input vector for the next iteration.
- α : The step size or learning rate, determining how far to push the input in the direction of the gradient at each step.
- $\nabla_{\mathbf{x}}$: The gradient operator with respect to the input $\mathbf{x}$.
- $\mathcal{L}$: The loss function used to evaluate the model's performance.
- $\mathbf{y}$: The ground-truth (true) label of the input $\mathbf{x}$.
- $f(\mathbf{x}_i)$: The model's prediction function evaluated at the current input $\mathbf{x}_i$.
- $\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{y}, f(\mathbf{x}_i))$: The gradient of the loss with respect to the input. It points in the direction that maximizes the loss for the true label $\mathbf{y}$, meaning adding this term actively degrades the model's performance on the true class.

2. What is the difference between a *targeted* and an *untargeted* attack? How would the formula change for a targeted attack?
- **Untargeted Attack:** The adversary's goal is simply to make the model predict *any* incorrect label. This is achieved by maximizing the loss with respect to the true label y (gradient ascent), as shown in the original formula.
 - **Targeted Attack:** The adversary aims to force the model to predict a specific, pre-determined incorrect class y_{target}
 - **Formula Change:** To make the model output y_{target} , the adversary minimizes the loss with respect to that target class (gradient descent). The formula changes to:

Original

$$x_{i+1} = x_i + \alpha \nabla_x \mathcal{L}(y, f(x_i))$$

vs

Changed Formula

$$x_{i+1} = x_i - \alpha \nabla_x \mathcal{L}(y_{\text{target}}, f(x_i))$$

- x_i (The Current Image): It starts with the current version of the image.
- The Target (y_{target}): The attacker picks a specific wrong label they want the AI to output (for example, making the AI see a **Stop Sign** but think it is a **Speed Limit Sign**).
- The Cheat Sheet ($\nabla_x \mathcal{L}$): This part calculates exactly how the image's pixels need to change to make the AI more confident in that wrong target answer.
- The Minus Sign ($-$): Because it uses a minus sign, it is performing "gradient descent" on the target error. This means it is actively **shrinking the AI's confusion** about the target class, pushing the AI closer and closer to making the exact mistake the attacker wants.
- The Step Size (α): It changes the pixels by just a tiny, microscopic amount so a human won't notice the difference.
- x_{i+1} (The New Image): The result is a brand-new version of the image that looks identical to a human, but completely fools the AI.

- **An Everyday Analogy**

Imagine you are trying to forge a signature. You look at the target signature, pick up your pen, and make a tiny stroke. Then you look again, adjust your hand slightly to match the target better, and make another stroke.

This formula does the exact same thing: it adjusts an image's pixels bit-by-bit until the AI is completely convinced it is looking at the target object.

3. Why does this basic update rule does not respect a **perturbation budget** $\|x_0 - x_t\| \leq \epsilon$?

How can it be modified to do so?

In simple terms, a **perturbation budget** is a **strict "spending limit"** for how much you are **allowed to tweak or change an input** (like an image or data).

Think of it like this:

- **The Goal:** You want to add just enough subtle edits or noise to test an AI's limits or trick it.
- **The Limit (The Budget):** You must stay under a certain line. If you change the data too much, the edits become completely obvious to a human, or the whole system breaks.

It is a mathematical boundary that ensures your changes stay **invisible and controlled** while you test how tough the AI truly is.

$\|x_0 - x_t\| \leq \epsilon$ -> ensures the changes remain imperceptible to humans while potentially fooling an AI model.

Why it fails to respect the budget: The basic update rule iteratively adds unconstrained gradient steps. If the gradient magnitude is large or the process is run for multiple steps, the total distance between the original input x_0 and the perturbed input x_t can easily grow larger than the allowed budget ϵ ($\|x_0 - x_t\| > \epsilon$). [PDF + 2](#)

How to modify it: To keep the perturbation within bounds, a projection operator (Proj) must be applied after each step. This technique is known as **Projected Gradient Descent (PGD)**. The modified formula is:

[PDF](#)

$$x_{i+1} = \text{Proj}_{x_0, \epsilon} (x_i + \alpha \nabla_x \mathcal{L}(y, f(x_i)))$$

where $\text{Proj}_{x_0, \epsilon}$ clips or projects the updated input back into an ϵ -ball centered around the original clean input x_0 . [PDF](#)

Original Formula $x_{i+1} = x_i + \alpha \nabla_x \mathcal{L}(y, f(x_i))$

Exercise 8.3: Defenses

Explain the idea of *adversarial training*. What trade-off does it introduce?

In simple words, **adversarial training** is like giving an AI a practice test full of trick questions while it is still learning.

Here is how it works:

- **The Setup:** Instead of only teaching the AI with perfect, normal data, you deliberately create "hacked" examples and mix them right into its daily lessons.
- **The Practice:** The AI is forced to figure out these tough, worst-case scenarios instead of just cruising through the easy stuff.
- **The Result:** By learning to handle these traps early on, the AI hardens its logic and becomes much harder to fool by sneaky, subtle changes in the real world.

Self-Driving Cars Traffic Sign Recognition

- **The Threat:** Attackers discovered that putting specific, mathematically designed stickers or tape on a **Stop sign** could completely fool a self-driving car. To a human, it just looks like minor graffiti, but to the car's camera AI, the stickers alter the pixels just enough to make it see a **45 MPH Speed Limit sign**.
- **The Adversarial Training:** Companies like Tesla or Waymo use adversarial training by digitally generating millions of variations of these "sticker attacks" on road signs and feeding them to the AI during its training phase. The car learns to ignore the random tape and focus on the actual geometry and shape of the sign so it doesn't accidentally speed through an intersection.

Trade-offs

1. **Robustness vs. Clean Accuracy:** The most prominent trade-off is a drop in standard accuracy. Models trained to be robust against adversarial attacks frequently perform slightly worse on perfectly clean, unperturbed data.
2. **Computational Complexity:** Generating high-quality adversarial examples (like multi-step PGD examples) inside the training loop drastically increases the time and computational resources required to train the model compared to standard training.

Practical: Attacking the CARLA Model

Exercise 8.4: Generating Adversarial Examples

Load your trained CARLA model and generate adversarial examples using the Fast Gradient Sign Method (FGSM):

$$x_{\text{adv}} = x + \varepsilon \cdot \text{sign}^1 \nabla_x \mathcal{L}(y, f(x))^2 \quad (2)$$

1. Implement FGSM for each of your three binary classifiers.
2. Generate adversarial examples for $\varepsilon \in \{0.01, 0.05, 0.1\}$.
3. Display a clean image and its adversarial counterpart side-by-side. At what ε do the perturbations become visible to a human?

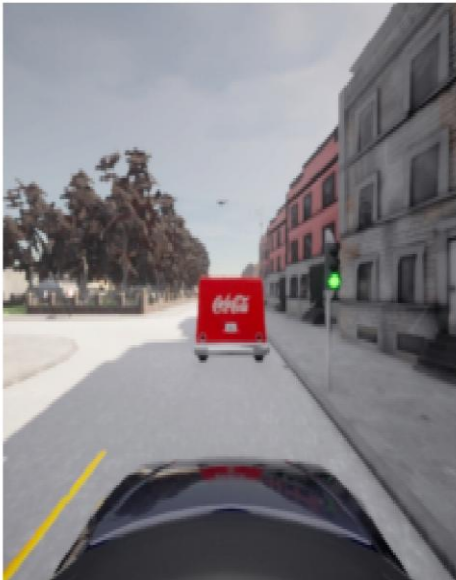
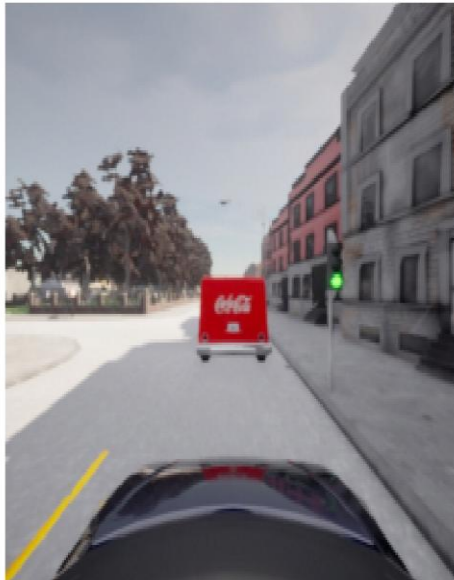
Pedestrian Model Examples :



Clean (Label=1)

Adversarial ($\epsilon=0.01$)

Clean (Label=0)

Adversarial ($\epsilon=0.01$)

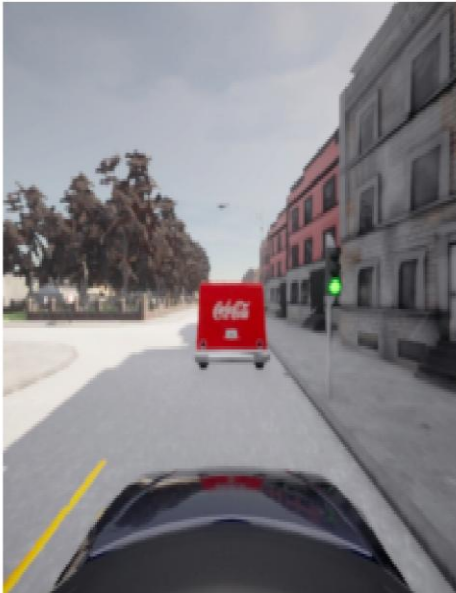
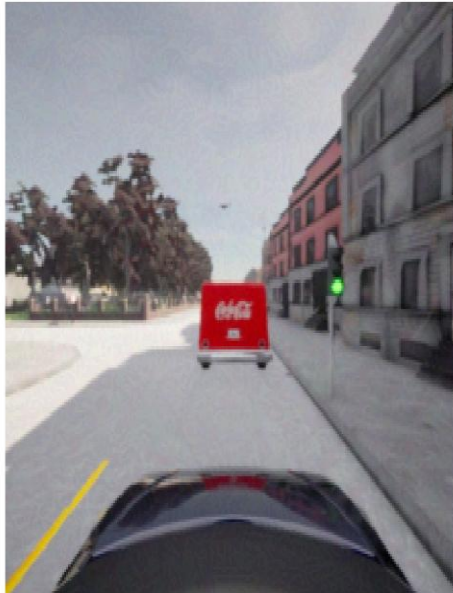
Clean (Label=1)

Adversarial ($\epsilon=0.01$)

Clean (Label=1)

Adversarial ($\epsilon=0.05$)

Clean (Label=0)

Adversarial ($\epsilon=0.05$)

Clean (Label=1)

Adversarial ($\epsilon=0.05$)

Exercise 8.5: Measuring Robustness

1. Evaluate each of your three models on the adversarial test set for each ϵ . (You may also use a subset of 100 randomly sampled images for evaluation.) Report the per-model recall drop compared to clean inputs.

EXERCISE 8.5 SUMMARY: ROBUSTNESS EVALUATION

=====							
EXERCISE 8.5 SUMMARY: ROBUSTNESS EVALUATION							
=====							
Model	Epsilon	Clean Recall	Adversarial Recall	Recall Drop	Drop %		
Pedestrian	0.01	0.5200	0.2200	0.3000	57.7%		
Pedestrian	0.05	0.5200	0.0900	0.4300	82.7%		
Pedestrian	0.10	0.5200	0.0700	0.4500	86.5%		
Traffic_light	0.01	0.9900	0.6500	0.3400	34.3%		
Traffic_light	0.05	0.9900	0.4700	0.5200	52.5%		
Traffic_light	0.10	0.9900	0.3000	0.6900	69.7%		
Vehicle	0.01	0.8300	0.6600	0.1700	20.5%		
Vehicle	0.05	0.8300	0.5400	0.2900	34.9%		
Vehicle	0.10	0.8300	0.4800	0.3500	42.2%		

KEY FINDINGS**PEDESTRIAN:**

Clean recall: 0.5200

$\epsilon=0.01$: 0.2200 recall (drop: 57.7%) [SEVERE]

$\epsilon=0.05$: 0.0900 recall (drop: 82.7%) [SEVERE]

$\epsilon=0.1$: 0.0700 recall (drop: 86.5%) [SEVERE]

TRAFFIC_LIGHT:

Clean recall: 0.9900

$\epsilon=0.01$: 0.6500 recall (drop: 34.3%) [HIGH]

$\epsilon=0.05$: 0.4700 recall (drop: 52.5%) [SEVERE]

$\epsilon=0.1$: 0.3000 recall (drop: 69.7%) [SEVERE]

VEHICLE:

Clean recall: 0.8300

$\epsilon=0.01$: 0.6600 recall (drop: 20.5%) [MODERATE]

$\epsilon=0.05$: 0.5400 recall (drop: 34.9%) [HIGH]

$\epsilon=0.1$: 0.4800 recall (drop: 42.2%) [HIGH]

OVERALL STATISTICS

Average recall drops across all models/epsilons: 53.5%

Maximum recall drop: 86.5%

Minimum recall drop: 20.5%

Recall drop by epsilon:

$\epsilon=0.01$: 37.5% (avg), 57.7% (max)

$\epsilon=0.05$: 56.7% (avg), 82.7% (max)

$\epsilon=0.1$: 66.1% (avg), 86.5% (max)

Exercise 8.6: Extending the Safety Analysis for Adversarial Robustness

You have now measured how much adversarial perturbations degrade your models' recall. Revisit your System-Theoretic Process Analysis from Sheet 2 and extend it to cover this threat. Many of you will not yet have an adversarial-attack entry in your analysis; the goal of this exercise is to add one and trace it through.

1. **Hazard.** Check the hazards from Exercise 2.4. Is a degraded perception output caused by an adversarial input captured (e.g. as a cause of “vehicle fails to brake for a pedestrian”)? If your hazard list does not reflect it, refine or add a hazard.

ID	Hazard	Losses	Likelihood	Severity
H-1	Pedestrian in path while vehicle is moving.	L-1, L-2.	Medium	High
H-2	Autonomous mode remains active outside the ODD, or operates on undetected out-of-distribution (OOD) visual inputs, leading to untrustworthy perception outputs.	L-1, L-2, L-3.	High	<i>High.</i>
H-3	Approaching intersection without detecting stop condition.	L-1, L-2, L-3	Medium	High
H-4	Planner commands acceleration when obstacle is in path.	L-1, L-2.	Low	High
<i>H5 (NEW)</i>	<i>Autonomous mode operates on undetected, adversarially perturbed visual inputs, leading to corrupted, high-confidence false perception outputs.</i>	<i>L-1, L-2, L-3</i>	<i>Medium</i>	<i>High</i>

2. **Unsafe control action.** Review the UCAs from Exercise 2.6. Add a UCA for the case where the planner acts on perception output that is wrong because the input has been adversarially perturbed and this is not detected (e.g. “the planner continues at speed while the pedestrian classifier has been fooled by an adversarial attack and outputs a false negative”). Link it to the hazard(s) above.

ID	Controller	Control Action	UCA Type	Hazard	Unsafe Scenario
UCA-1	Planner	Continue Driving	Provided unsafely	H-1	Pedestrian detected in path but vehicle does not stop.
UCA-2	Planner	Emergency Brake	Not provided	H-1	Pedestrian within critical distance but brake not issued.
UCA-3	Planner	Emergency Brake	Wrong timing	H-3	Braking issued too late to stop at a detected intersection.
UCA-4	Operator	Manual Override	Not provided	H-2	Automation fails outside ODD; operator fails to take over.
UCA-5	Planner	Emergency Brake	Provided unsafely	H-4	Hard brake issued on highway with no obstacle, causing rear-end.
UCA-6	Planner	Request Decel	Not provided	H-3	Traffic light detected (presence) but no slowdown initiated.
UCA-7	Operator	Manual Override	Wrong timing	H-1	Operator reacts too late ($> 1.5s$) to a missed ML detection.
UCA-8	Planner	Continue Driving	Wrong timing	H-3	Continues through intersection after light presence was detected.
UCA-9	Planner	Continue Driving	Provided unsafely	H-2, H-1, H-3	The planner continues driving at nominal operational speed while the camera input is out-of-distribution (e.g., heavy fog or night) and the perception outputs are untrustworthy because the distribution shift has gone undetected.
UCA - 10	Planner	Continue Driving/Accelerate	Provided Unsafely	H-5, H-1, H-3, H-4	The planner continues driving at operational speed or commands acceleration because a successful adversarial attack has silently hidden an obstacle (false negative), causing the system to act as if the path is clear.

3. **Safety constraints.** From that UCA, derive at least one new safety constraint of each kind (cf. Exercise 2.7):

- a **model-level** constraint on adversarial robustness, referencing the ϵ budget and recall drop you measured; and

Model-Level Constraint (Adversarial Robustness)

This requirement defines the mathematical resilience boundaries your classifiers must satisfy based on the testing parameters from Exercise 8.4 and 8.5.

Constraint: Constraint: The pedestrian, vehicle, and traffic light classifiers' recall drop under adversarial perturbation must be empirically characterized across the target perturbation budget $\|x_0 - x_t\| \leq \epsilon$ for $\epsilon \in \{0.01, 0.05, 0.1\}$, and minimized through defensive measures (e.g., adversarial training) rather than assumed acceptable in advance. Current undefended measurements (Exercise 8.5) show recall drops of **20.5%–86.5%** across all three models even at the smallest tested $\epsilon=0.01$, which is **not acceptable** for deployment and establishes the baseline this constraint requires improvement against.

- a **system-level** constraint on the response when a low-confidence or anomalous prediction is detected.

System Level Constraint (Fail Safe Response)

Because ML models can never achieve 100% adversarial immunity, an architectural constraint handles cases where an attack manages to successfully degrade model certainty. **Constraint:** If runtime defense monitors (e.g., feature-space anomaly checkers or input-reconstruction filters) flag a visual frame as an unvalidated or suspected adversarial manipulation, or if model confidence breaks calibration, the Planning Module must immediately inhibit normal speed or acceleration commands.

* **System Action:** The system must initiate an automated Minimum Risk Maneuver (MRM), commanding an immediate controlled deceleration profile to safely transition the vehicle into a stop state within 1.5 seconds, while issuing a critical takeover prompt to the operator.

4. **Residual risk.** Even with adversarial training that meets your model-level constraint, what risk remains? Does robust training fully address the UCA and hazard?

- **Vulnerability to Advanced and Adaptive Attacks:** Adversarial training optimized against basic gradient attacks like the Fast Gradient Sign Method (FGSM) does not guarantee safety against iterative attacks (such as PGD), black-box optimization vectors, or structural changes.
- **Physical-World Vector Gaps:** Digital pixel-level defense models ($\|x_0 - x_t\| \leq \epsilon$) offer very little protection against physical-world semantic hacks, such as strategically placed adversarial tape/stickers on traffic signs or physical road markings designed to trigger false negatives.
- **The Performance Trade-Off:** Maximizing adversarial robustness frequently distorts a network's standard decision boundaries. This introduces a residual risk where the model

becomes overly conservative, generating an increased rate of standard false positives or false negatives under completely nominal, unattacked conditions.

- **Lack of Complete Mitigation:** Robust training merely reduces the probability of a singular model failure mode. It fails to structurally resolve the UCA or high-level hazards on its own. To build an acceptable safety case, the vehicle must rely on deep engineering redundancy—such as cross-modal sensor fusion (verifying camera feeds against unattacked LiDAR/Radar ranges) and temporal verification steps

